

Supplementary Computational Appendix

Set-Valued Causal Inheritance for Functional Self-Continuity

Author: Hongju Liu

Report: TA-TR-2026-17

DOI: 10.5281/zenodo.22950904

Companion to: Final Preprint Manuscript v1.0

Date: 25 September 2026

This supplement contains the detailed computational protocols, secondary benchmark tables, ablations, negative results, and analytical appendices moved out of the main manuscript. It is intentionally more exhaustive than the main paper. The same evidence hierarchy and non-claims apply.

The preceding sections establish exact representation limits. This section tests whether those limits remain visible when the representations are learned from finite synthetic data.

These experiments are **controlled stress tests**. Their purpose is to verify computational consequences of the theory under known ground truth. They are not empirical evidence for human selfhood.

All reported models are intentionally simple logistic regressions. This prevents performance gains from being attributed to large-model capacity.

S1. Benchmark A: matched singleton marginals, different joint successor structure

We construct a binary context variable C .

Context 0 — exclusive continuation

Exactly one candidate is a successor:

$$P(R = \{B\} \mid C = 0) = 0.5,$$

$$P(R = \{C\} \mid C = 0) = 0.5.$$

Context 1 — fission-or-death

Either both candidates are successors or neither is:

$$P(R = \emptyset \mid C = 1) = 0.5,$$

$$P(R = \{B, C\} \mid C = 1) = 0.5.$$

In both contexts:

$$P(B \in R \mid C) = P(C \in R \mid C) = 0.5.$$

Thus the context contains **no information about either singleton marginal**, but it perfectly determines the dependency structure.

Models

Independent model. Two separate logistic regressions predict $B \in R$ and $C \in R$, and their probabilities are multiplied to produce a joint distribution.

Joint successor-set model. One multinomial logistic regression predicts the four possible set states:

$$00, \quad 10, \quad 01, \quad 11.$$

The training and test sets were exactly balanced across the two possible outcomes in each context, eliminating finite-sample marginal leakage.

Results

Model	Test NLL (bits/episode)	Singleton/joint- state accuracy	Cardinality- sensitive decision reward
Independent Bernoulli	2.000	~ 0.50	0.50
Joint successor- set	1.000	~ 0.50	1.00

The joint-state classification accuracy is approximately 0.50 for both models because each context intentionally leaves one binary uncertainty unresolved. Accuracy is therefore not the relevant metric.

The important result is probabilistic calibration:

$$\Delta\text{NLL} \approx 1.000 \text{ bit/episode},$$

matching the exact total-correlation lower bound.

The decision task asks the model to choose between:

$$a_{\text{one}}$$

which is rewarded iff $|R| = 1$, and:

$$a_{\text{nonone}}$$

which is rewarded iff $|R| \in \{0, 2\}$.

Because the independent model maps both contexts to the same marginal vector:

$$(Q_B, Q_C) = (0.5, 0.5),$$

it cannot distinguish the two decision states and obtains mean reward 0.5.

The joint successor-set model uses the context to distinguish:

$$\{10, 01\}$$

from:

$$\{00, 11\},$$

and obtains reward 1.0.

This computational result exactly instantiates Theorems 1 and 2.

S2. Benchmark B: endpoint similarity versus causal-lineage evidence

The second benchmark targets the difference between:

endpoint resemblance

and:

live causal provenance.

Two candidate future processes B, C are presented. Exactly one is the live successor.

Training distribution

The true successor is normally highly similar to the current state:

$$\mu_{\text{sim}}^{\text{live}} = 0.90,$$

while the non-successor has mean similarity:

$$\mu_{\text{sim}}^{\text{detached}} = 0.20.$$

A **similarity model** is trained only on the two endpoint-similarity features.

A **causal-evidence model** is trained only on two noisy provenance features. The live successor has mean causal evidence 0.72 and the detached candidate 0.28, both with standard deviation 0.18 before clipping to $[0, 1]$.

OOD transformation

A severity parameter:

$$\alpha \in [0, 1]$$

progressively reverses the similarity cue:

$$\mu_{\text{sim}}^{\text{live}}(\alpha) = 0.90 - 0.65\alpha,$$

$$\mu_{\text{sim}}^{\text{detached}}(\alpha) = 0.20 + 0.75\alpha.$$

The causal evidence distribution is held fixed.

At:

$$\alpha = 0.5,$$

the two candidates have equal expected similarity.

At:

$$\alpha = 1,$$

the live successor is deliberately transformed to low similarity while the detached candidate is a high-similarity lookalike.

Results

OOD severity α	Similarity accuracy	Causal- evidence accuracy	Similarity NLL (bits)	Causal NLL (bits)
0.00	1.000	0.957	0.002	0.163
0.25	0.999	0.957	0.065	0.159
0.50	0.490	0.957	1.251	0.156
0.75	0.001	0.953	5.488	0.170
1.00	0.000	0.957	10.666	0.151

The similarity model generalizes only while the training correlation between resemblance and lineage remains valid.

At the crossover:

$$\alpha = 0.5,$$

its accuracy becomes approximately chance.

After the correlation reverses, the model becomes confidently wrong.

The causal-evidence model is imperfect because its evidence is deliberately noisy, but its accuracy remains approximately:

$$95\% - 96\%$$

across the OOD sweep.

Interpretation

This result should not be read as the claim that biological identity can be solved by a two-feature provenance classifier.

It establishes the narrower computational point:

A representation trained on endpoint similarity can fail catastrophically under transformations that preserve causal continuation while reversing resemblance.

The benchmark therefore operationalizes a distinction long emphasized in philosophical continuity discussions, but tests it as an OOD representation problem.

S3. Reproducibility details

Benchmark A uses exactly balanced synthetic samples in each context so that singleton marginals are exactly 0.5 in both train and test sets.

Benchmark B uses:

- 40,000 training episodes;
- 15,000 test episodes per OOD severity;
- logistic regression classifiers;
- Gaussian cue noise with clipping to $[0, 1]$;
- a fixed pseudorandom seed in the accompanying implementation.

No neural network or large language model is required to obtain the reported effects.

The simplicity is intentional: the benchmarks test representational restrictions rather than model capacity.

S4. What these benchmarks do and do not establish

They establish that:

1. the one-bit independent-model penalty is visible in finite-data learning;
2. marginal-only representations can be decision-insufficient even when their marginals are perfectly calibrated;
3. endpoint similarity can be an OOD-fragile surrogate for causal lineage.

They do **not** yet establish that:

- biological humans use successor-set posteriors;
- artificial agents naturally learn Γ ;
- the MECS state is identifiable in realistic neural networks;
- the framework predicts phenomenal consciousness.

The next empirical step is a multi-scenario benchmark in which the same learned model must jointly handle continuation, detached copying, redundant fission, complementary splitting, synergy, fusion, and restoration.

S5. Unified nine-scenario benchmark

The previous two benchmarks isolate one theoretical failure at a time. We next train the model family on a single mixed environment containing nine scenario types:

1. termination;
2. ordinary continuation;
3. detached clone;
4. full fission;
5. complementary split;
6. secret-sharing split;
7. fusion;
8. radical transformation;
9. backup restoration.

The training set contains 90,000 episodes and the in-distribution test set contains 45,000 episodes. Candidate labels B/C are randomized in one-successor scenarios.

The observable synthetic cues are deliberately separated into levels:

- endpoint similarity;
- noisy individual provenance evidence;
- a noisy joint-causal probe;
- a noisy interaction probe.

The hierarchy therefore tests **representational sufficiency under controlled access to evidence**. It does not claim that a biological or deployed AI system is naturally given such probes.

The six model classes are:

- **T0**: similarity-only joint successor classifier;
- **T1**: one-successor categorical representation, structurally unable to represent 11 fission states;

- **T2**: independent Bernoulli successor field;
- **T3**: joint successor-set posterior using individual similarity and provenance evidence;
- **T4**: set-aware causal model with a noisy joint-inheritance probe;
- **T5**: interaction-aware model with an additional noisy pair-interaction probe.

For decision evaluation, preserving one future process costs 0.2, preserving both costs 0.4, and utility depends on the true inheritance set function:

$$u(\emptyset) = 0,$$

$$u(\{B\}) = \Gamma(B) - 0.2,$$

$$u(\{C\}) = \Gamma(C) - 0.2,$$

$$u(\{B, C\}) = \Gamma(B, C) - 0.4.$$

In-distribution results

Model	Set NLL (bits)	Exact set accuracy	Representational coverage	Γ MAE	Mean decision regret	Optimal action match
T0	1.550	0.506	1.000	0.307	0.252	0.383
T1	∞ / unsupported	0.664	0.664	0.137	0.096	0.746
T2	0.526	0.820	1.000	0.186	0.135	0.626
T3	0.206	0.925	1.000	0.182	0.142	0.627
T4	0.005	0.999	1.000	0.049	0.005	0.926
T5	0.004	0.999	1.000	0.047	0.003	0.933

Three observations matter.

First, **T1 is structurally incomplete**, not merely inaccurate. Approximately one third of this balanced scenario mixture contains genuine two-successor states; T1 has no finite probability assignment to them, so its full-set NLL is undefined/infinite even though some downstream actions happen to be adequate.

Second, moving from T2 to T3 substantially improves successor-set prediction:

$$0.526 \rightarrow 0.206$$

bits/episode in test NLL. However, T3’s decision regret does not improve correspondingly. In fact, it remains around 0.142. This is expected: T3 learns **which future processes are successors**, but it does not model how functional inheritance is distributed across those successors.

Third, T4 and T5 sharply reduce decision regret:

$$0.0050$$

and:

$$0.0033,$$

respectively. This demonstrates the conceptual distinction between:

successor-set belief

and:

set-valued causal inheritance.

A model may know that B and C are both genuine successors yet still make the wrong preservation decision if it cannot distinguish full redundancy from complementary or synergistic inheritance.

S6. Scenario-held-out generalization

To test whether the models merely memorize scenario labels, each scenario is removed entirely from training and then evaluated as an unseen test regime.

The most diagnostic held-out cases are:

Held-out scenario	Model	Exact set accuracy	Mean decision regret
detached_clone	T3	1.000	0.000
detached_clone	T4	1.000	0.007
detached_clone	T5	1.000	0.004
complementary	T3	0.985	0.300
complementary	T4	0.997	0.008
complementary	T5	0.997	0.003
secret_share	T3	0.000	0.600
secret_share	T4	0.804	0.000
secret_share	T5	0.730	0.000
fusion	T3	0.898	0.000
fusion	T4	0.986	0.013
fusion	T5	0.985	0.012

The **complementary** condition is especially revealing. T3 generalizes the successor set itself with approximately:

98.5%

accuracy, yet its decision regret remains:

0.300.

It knows that both B and C are successors, but under the successor-only surrogate it still treats preserving one as sufficient. T4 and T5 retain similar set accuracy while reducing regret to approximately 0.008 and 0.003.

The **secret-share** condition is more extreme. When this scenario is completely absent from training, T3 fails to recognize the joint-successor structure and incurs roughly 0.600 decision regret. T4 and T5, which receive set-level causal evidence, achieve zero decision regret in this controlled benchmark even though their exact set-classification accuracy is not perfect.

These are representational stress tests, not claims that real systems receive an oracle “joint probe.” The result shows what additional evidence is required by the decision problem.

S7. Full-mixture OOD similarity reversal

The endpoint-similarity relationship is then progressively reversed in the relevant one-successor scenarios while the causal evidence channels remain stable.

At maximum reversal, the results are:

Model	Set NLL (bits)	Exact set accuracy	Mean decision regret
T0	1.939	0.461	0.419
T1	∞ / unsupported	0.665	0.096
T2	0.526	0.820	0.135
T3	0.210	0.923	0.141
T4	0.006	0.999	0.005
T5	0.006	0.999	0.003

The similarity-only model’s mean decision regret rises from approximately 0.252 in-distribution to 0.419 under maximum reversal. Models with explicit provenance remain comparatively stable.

This does not demonstrate that “causal evidence” is easy to obtain in practice. It demonstrates that if a system uses resemblance as a surrogate for lineage, then a distribution shift that separates resemblance from lineage is a direct failure mode.

S8. Interpretation of the unified benchmark

The benchmark supports three nested claims.

Representation Claim A — Multi-successor support is necessary

T1 fails structurally whenever the true successor set has cardinality greater than one.

Representation Claim B — Joint successor dependence is decision-relevant

T2 loses information by factorizing the successor indicators; Theorem 2 quantifies the irreducible penalty by total correlation.

Representation Claim C — Successor identity and inherited function are different targets

T3 can identify a joint successor set yet still choose the wrong preservation action when it does not know whether those successors are redundant, complementary, or synergistic.

Therefore the framework requires both:

$$\mu(R)$$

and:

$$\Gamma(S).$$

The near-perfect T4/T5 in-distribution scores should **not** be interpreted as a discovery result. The synthetic data generator deliberately exposes noisy joint-causal and interaction probes constructed from known ground-truth inheritance quantities. T4/T5 are therefore **oracle-proxy sufficiency checks**: they test whether the representation can use the information identified by the theory, not whether that information can be discovered from raw observations. These scores are not used as independent empirical evidence for the framework.

The later trajectory-based experiments remove these oracle-proxy features and estimate $\Gamma(S)$ from randomized process responses.

S9. Estimating set-level inheritance from intervention trajectories

The preceding unified benchmark gave T4/T5 engineered noisy measurements of joint causal inheritance. That experiment verified representational sufficiency but left an important gap:

Could the set-level inheritance variables be estimated from intervention trajectories rather than supplied as features?

We therefore construct a second synthetic benchmark in which no scenario label, true Γ , or engineered joint-causal probe is given to the learner.

Intervention protocol

Each episode contains a four-bit current endogenous state.

For that episode, the system is re-run under:

$$N_{\text{int}} = 96$$

randomized interventions on the four current bits.

The same future process-generation mechanism is evaluated under every intervention.

For each current bit U_j , we estimate finite-sample mutual information with:

- future candidate B's corresponding response bit;
- future candidate C's corresponding response bit;
- the joint two-candidate response pair.

A two-permutation shuffle baseline is subtracted to reduce finite-sample upward bias.

The resulting per-episode estimates are:

$$\hat{\Gamma}(B), \quad \hat{\Gamma}(C), \quad \hat{\Gamma}(B, C).$$

The benchmark uses 12,000 training episodes and 6,000 test episodes with a fixed pseudorandom seed.

A 2% bit-flip probability is applied to future outputs so that the estimated inheritance is not trivially deterministic.

Ten scenario types

The benchmark includes:

1. termination;
2. ordinary continuation;
3. detached clone;
4. full fission;
5. redundant half-copy;
6. complementary split;
7. secret-sharing split;
8. fusion;
9. radical transformation;
10. backup restoration.

The redundant-half condition is added specifically to create a matched-singleton inheritance contrast with complementary splitting:

$$\Gamma(B) \approx \Gamma(C) \approx 0.5$$

in both cases, while:

$$\Gamma(B, C) \approx 0.5$$

for redundant inheritance and:

$$\Gamma(B, C) \approx 1$$

for complementary inheritance.

Estimated inheritance signatures

The mean trajectory-derived estimates are:

Scenario	$\hat{\Gamma}(B)$	$\hat{\Gamma}(C)$	$\hat{\Gamma}(B, C)$	$\hat{m}_{\Gamma}(BC)$
Termination	0.005	0.004	0.009	-0.001
Full fission	0.863	0.861	0.944	-0.780
Redundant half-copy	0.431	0.433	0.475	-0.388
Complementary split	0.432	0.433	0.858	-0.007
Secret-share	0.005	0.004	0.765	+0.756

The estimated interaction is:

$$\hat{m}_{\Gamma}(BC) = \hat{\Gamma}(B, C) - \hat{\Gamma}(B) - \hat{\Gamma}(C).$$

Despite finite-sample bias, output noise, and randomized episode generation, the estimator recovers the intended qualitative structures:

- strongly negative interaction for redundant copying;
- approximately additive interaction for complementary splitting;
- strongly positive interaction for secret-sharing succession.

The key comparison is:

redundant-half vs. complementary.

Their singleton estimates are nearly identical:

$$\hat{\Gamma}(B) \approx \hat{\Gamma}(C) \approx 0.43,$$

but their joint estimates differ sharply:

0.475 vs. 0.858.

Thus the difference is recovered from actual randomized intervention trajectories rather than from an oracle scenario label.

Learned model results

The same T0-T5 representation hierarchy is then trained.

Model	Set NLL (bits)	Set accuracy	Mean decision regret	Optimal- action match
T0 — similar- ity only	1.552	0.435	0.220	0.166
T1 — one- successor categori- cal	20.107	0.597	0.096	0.706
T2 — independ- ent Bernoulli	0.348	0.926	0.095	0.664
T3 — joint suc- cessor set using singleton inheri- tance	0.177	0.933	0.125	0.610

Model	Set NLL (bits)	Set accuracy	Mean decision regret	Optimal- action match
T4 — set- aware using trajectory- estimated joint inheri- tance	0.0049	1.000	0.000	0.901
T5 — interaction- aware	0.0048	1.000	0.000	0.901

The most important result is not the near-perfect set classification of T4/T5. The data generator is intentionally designed so that the intervention-derived causal structure is diagnostic.

The key result is the transition from:

successor membership

to:

set-level functional inheritance.

T3 can usually infer whether B and C are members of the successor set, yet its planning regret remains nonzero because it cannot distinguish how much functionality is jointly preserved.

Scenario-specific decision results

The complementary and secret-sharing conditions isolate this failure.

For complementary splitting:

T3 regret = 0.300,

while:

T4/T5 regret = 0.

For secret-sharing continuation:

$$\text{T3 regret} \approx 0.796,$$

while:

$$\text{T4/T5 regret} = 0.$$

In the complementary case, T3 correctly predicts:

$$R = \{B, C\},$$

but its successor-membership representation still treats preserving one successor as sufficient. The intervention-derived set inheritance correctly identifies that preserving the pair is required to retain the full current functional state.

In the secret-sharing case, singleton inheritance is approximately zero. The high joint estimate:

$$\hat{\Gamma}(B, C) \approx 0.765$$

is what makes pair preservation rational.

Why optimal-action match is below one despite zero regret

T4/T5 obtain zero measured regret but an optimal-action match of approximately 0.90.

This is not contradictory.

Some scenarios admit multiple utility-maximizing actions. For example, under full redundant copying, preserving either full successor can have the same utility. The action-match metric chooses one arbitrary argmax from the tied optimum set; regret correctly treats all equal-utility actions as optimal.

Decision regret is therefore the appropriate primary metric.

Important limitations of the trajectory estimator

This experiment removes the engineered joint-causal probe, but it is still deliberately simplified.

1. The current endogenous state is already presented in a known four-bit functional basis.

2. Intervention coordinates and future response coordinates are aligned.
3. Mutual information is estimated bitwise and averaged.
4. Candidate future processes are supplied by the environment.
5. The intervention policy is uniform and externally controlled.

Therefore this controlled experiment demonstrates the following pipeline:

intervention $\rightarrow$ set-level causal-inheritance estimate $\rightarrow$ better preservation decisions

in a controlled system.

It does **not** yet solve the harder MECS identification problem in a high-dimensional learned agent.

A stronger benchmark should infer a representation-invariant minimal endogenous state from raw recurrent activations, intervene in that state, and test whether the resulting set-valued inheritance estimate is stable under invertible re-encoding.

S10. Representation-invariance benchmark

The v0.5 intervention estimator averaged coordinate-aligned per-bit mutual information. That estimator is useful in a known aligned code but it is **not representation invariant**.

To test this directly, we compare:

1. a **joint-state estimator** using full discrete states:

$$\Gamma(B) = I(U; B)/H(U),$$

$$\Gamma(C) = I(U; C)/H(U),$$

$$\Gamma(B, C) = I(U; B, C)/H(U);$$

2. the previous coordinate-aligned bitwise estimator.

Protocol

We use four-bit intervention states and six inheritance structures:

- ordinary continuation;
- radical invertible transformation;
- full fission;
- redundant half-copy;
- complementary split;
- secret-sharing split.

Each of the 16 intervention states is repeated 128 times. Future outputs receive 2% independent bit-flip noise.

For each scenario we then apply:

200

independent random bijections to:

- the 16 intervention-state labels;
- B's 16-state encoding;
- C's 16-state encoding.

The causal system and samples are unchanged. Only the reversible representation labels change.

Results

	Joint $\Gamma(B)$	Joint $\Gamma(C)$	Joint $\Gamma(B, C)$	Bitwise B	Bitwise C	Bitwise BC
Scenario	mean $\pm$ sd	mean $\pm$ sd	mean $\pm$ sd	mean $\pm$ sd	mean $\pm$ sd	mean $\pm$ sd
complementary	0.4508 $\pm$ 5.3e-17	0.4497 $\pm$ 3.9e-17	0.9238 $\pm$ 1.0e-16	0.0082 $\pm$ 0.0007	0.0039 $\pm$ 0.0003	0.0038 $\pm$ 0.0104
full_fission	0.8692 $\pm$ 1.0e-16	0.8667 $\pm$ 1.0e-16	0.9700 $\pm$ 8.6e-17	0.0391 $\pm$ 0.0025	0.0355 $\pm$ 0.0033	0.0338 $\pm$ 0.0481
ordinary	0.8558 $\pm$ 1.0e-16	0.9187 $\pm$ 7.8e-19	0.9112 $\pm$ 6.4e-17	0.0406 $\pm$ 0.0020	0.0393 $\pm$ 0.0002	0.00213 $\pm$ 0.0295
radical_transform	0.8603 $\pm$ 9.0e-17	0.9207 $\pm$ 1.0e-18	0.9127 $\pm$ 5.0e-17	0.0403 $\pm$ 0.0020	0.0384 $\pm$ 0.0003	0.0031 $\pm$ 0.0283
redundant_half	0.4385 $\pm$ 2.0e-17	0.4462 $\pm$ 6.0e-17	0.5429 $\pm$ 7.0e-17	0.0086 $\pm$ 0.0006	0.0069 $\pm$ 0.0007	0.0078 $\pm$ 0.0111
secret_sharing	0.0218 $\pm$ 1.6e-18	0.0197 $\pm$ 3.6e-18	0.8525 $\pm$ 3.6e-17	0.0004 $\pm$ 0.0002	0.0002 $\pm$ 0.0002	0.00024 $\pm$ 0.0020

The joint-state estimator is unchanged to floating-point precision under every recoding, exactly as Theorem 1 predicts.

The coordinate-aligned bitwise estimator is not invariant. For ordinary continuation, for example, the full-state inheritance remains:

$$\Gamma(B) \approx 0.856,$$

while the same causal process, written under random reversible codes, has an average coordinate-aligned estimate of only:

$$0.041$$

with substantial recoding-to-recoding variation.

For secret-sharing succession, the full joint inheritance remains approximately:

$$\Gamma(B, C) \approx 0.853,$$

while the coordinate-aligned estimate collapses to approximately:

$$0.003.$$

Thus an inheritance estimator can be badly wrong even when the underlying causal system is literally unchanged.

This benchmark shows that coordinate-specific inheritance claims require independently justified coordinate semantics.

S11. MECS quotient-recovery benchmark

The previous benchmark assumes that the functional intervention variable U is already known. We next test the quotient-state principle itself.

System

The raw implementation state contains eight bits:

$$X = (Z, N),$$

where:

$$Z \in \{0, 1\}^4$$

is the four-bit functional state and:

$$N \in \{0, 1\}^4$$

is a pure nuisance state.

Hence there are:

$$2^8 = 256$$

raw implementation states.

Only Z affects endogenous future behavior. The nuisance bits never affect the declared endogenous outputs under any admissible intervention.

Intervention family

The admissible intervention set consists of:

- no intervention;
- flip functional bit 1;
- flip functional bit 2;
- flip functional bit 3;
- flip functional bit 4.

For each raw state, the complete intervention-response signature is the vector of future functional outputs under these five conditions.

Two raw states are placed in the same quotient class iff their complete signatures are identical.

Result

The quotient recovers exactly:

$$16$$

functional states from:

$$256$$

raw states.

Each quotient class contains:

raw states that differ only in nuisance bits.

Against the ground-truth four-bit functional state, the recovered partition has:

$$ARI = 1.000.$$

We then apply:

200

arbitrary bijective relabelings of the entire 256-state raw implementation space.

For every recoding:

- recovered MECS state count = 16;
- minimum class size = 16;
- maximum class size = 16;
- adjusted Rand index against the recoded ground-truth functional partition = 1.000.

This does not show that MECS recovery is easy in realistic neural systems. The benchmark is exact and finite-state.

It demonstrates the narrower structural claim:

if endogenous interventional equivalence is known, the quotient removes pure implementation nuisance and survives arbitrary raw-state relabeling.

S12. Consequence for the intervention estimator

v0.5 used a bitwise inheritance estimator because the synthetic functional coordinates were known.

v0.6 therefore changes the methodological hierarchy:

1. **primary theoretical object:** interventional channel $\mathcal{K}_{t \rightarrow S}$;
2. **representation-invariant finite-state summary:** full joint-state mutual information $\Gamma(S)$;
3. **coordinate-wise estimators:** permissible only when coordinate semantics are independently justified.

This distinction matters for any future application to learned neural representations.

The next benchmark should learn or approximate the MECS quotient from high-dimensional recurrent activations rather than being given exact equivalence signatures.

S13. Learning an approximate MECS from high-dimensional hidden states

The exact quotient benchmark in Supplementary Note S11 assumes access to complete intervention-response signatures for every finite raw state.

That assumption is unrealistic for learned agents.

We therefore test a weaker question:

Can an approximate endogenous functional quotient be learned from high-dimensional hidden states when each sample exposes only one intervention and the learner is never given the true functional-state label?

Hidden-state generator

The latent implementation state is:

$$X = (Z, N),$$

where:

- $Z \in \{0, 1\}^4$ is a four-bit functional state;
- $N \in \mathbb{R}^8$ is continuous nuisance.

The raw hidden representation has:

$$64$$

dimensions.

A fixed nonlinear feature map embeds (Z, N) into 64 dimensions, after which each benchmark run applies an independent random orthogonal transformation:

$$H^7 = H_0 Q_r.$$

Thus the raw coordinate system changes across runs while preserving all information.

We test:

8

independent recodings.

Incomplete intervention data

There are:

$$16 \times 5 = 80$$

possible functional-state/intervention combinations:

- no-op;
- flip functional bit 1;
- flip functional bit 2;
- flip functional bit 3;
- flip functional bit 4.

Sixteen combinations are removed entirely from training.

Every training episode provides only:

1. one 64-dimensional current hidden state;
2. one sampled four-bit intervention mask;
3. a twelve-dimensional future endogenous readout.

The learner never receives the true Z label.

Models

PredictiveBottleneck-4.

A neural encoder:

$$64 \rightarrow 64 \rightarrow 4$$

produces a four-dimensional code. The code and intervention mask predict the future endogenous readout.

Autoencoder-4.

The same four-dimensional bottleneck is optimized to reconstruct the 64-dimensional raw implementation state.

PCA-4.

Four principal components of the raw hidden representation.

RawPredictor-64.

A non-bottleneck future-readout predictor used only as a predictive-capacity reference.

In-distribution representation results

Across eight arbitrary recodings:

Representation	Functional-state accuracy	Linear nuisance R^2	Within-state variance / total variance
PredictiveBottleneck-4	1.000±0.000	-0.0002	0.0077
Autoencoder-4	0.090±0.005	0.444	0.992
PCA-4	0.091±0.003	0.493	0.993

The predictive code linearly decodes all sixteen functional states with essentially perfect accuracy:

$$1.000$$

while its linearly decodable nuisance content is approximately zero.

The within-functional-state variance ratio is only:

$$\approx 0.008,$$

showing that states differing only in nuisance collapse into tight functional clusters.

By contrast, the autoencoder and PCA codes retain roughly 44–49% of the nuisance variance under a linear probe and fail to organize their four-dimensional capacity around the functional quotient.

This is expected: nuisance coordinates were deliberately assigned greater raw variance than the four functional bits. Reconstruction and variance-preservation objectives therefore spend scarce representational capacity on implementation detail that does not affect endogenous future behavior.

Nuisance-distribution shift

At test time the nuisance distribution is shifted by:

$$+4$$

in every nuisance coordinate while the functional dynamics remain unchanged.

PredictiveBottleneck-4 retains functional-state accuracy:

$$0.99997$$

and future-readout MSE changes only from:

$$0.056$$

to:

$$0.064.$$

Autoencoder and PCA functional-state accuracies remain near chance-level values of roughly 0.09.

The negative nuisance R^2 values under this distribution shift are not interpreted as an information metric; linear R^2 is reported only as an in-distribution leakage diagnostic.

Held-out state×intervention combinations

The predictive bottleneck is then evaluated on the sixteen functional-state/intervention combinations that never occurred in training.

Its current functional-state decoding remains:

$$1.000.$$

However, future endogenous readout MSE rises from:

$$0.056$$

in-distribution to:

$$0.258.$$

The raw non-bottleneck predictor shows a similar transition-prediction penalty:

$$0.037 \rightarrow 0.213.$$

This separates two problems that should not be conflated:

state identification

and:

complete intervention-dynamics identification.

The learned four-dimensional state remains an accurate functional-state representation even when the transition predictor encounters intervention combinations outside its training support.

Why this benchmark matters

The benchmark does not establish a new causal-representation-learning algorithm.

Its significance for the present paper is narrower.

The theory requires a low-dimensional endogenous reference state that:

1. preserves distinctions relevant to the system’s own future operation;
2. discards pure implementation nuisance;
3. is robust to arbitrary reversible recoding of raw hidden coordinates;
4. can be approximated from incomplete intervention data.

The simple predictive bottleneck satisfies these requirements in this controlled generator substantially better than reconstruction-oriented baselines.

Remaining gap

The benchmark is still easier than a realistic persistent-agent problem.

The future endogenous readout is fixed and observed, the hidden-state mixing is information preserving, and the bottleneck dimensionality happens to equal the true functional dimension.

A stronger test should:

- learn the bottleneck dimension;
- use recurrent hidden states generated by a learned policy;
- permit partial observability;
- infer the intervention-relevant quotient without a pre-specified readout map;
- test whether set-valued inheritance computed on the learned state remains calibrated under long-horizon branch, merge, and restoration events.

S14. MECS approximation in a frozen persistent recurrent process

The previous learned-MECS benchmark used a high-dimensional but feed-forward synthetic hidden-state generator. We next ask whether a comparable quotient can be approximated from the hidden state of a genuinely recurrent process that has formed memory through interaction.

S14.1 Recurrent process

A GRU with 48 hidden units is trained to track a four-bit latent world state under partial observability.

At each time step:

- only one randomly selected world bit, or no bit, is observed;
- the observed bit is corrupted with 8% noise;
- eight unrelated distractor features are appended to the observation;
- an exogenous action can flip one of the four latent bits before the next step.

The recurrent system predicts all four current latent bits.

After 1,000 training updates, its mean bit accuracy is:

0.932

in-distribution and:

0.918

when the observation-nuisance mean is shifted by +5.

The recurrent process is then frozen.

S14.2 Raw implementation state

The raw persistent implementation snapshot is not defined as the GRU hidden state alone.

We append an independent 16-dimensional persistent cache/register that undergoes slow stochastic drift but is never supplied to:

- the GRU;
- the policy/readout;
- any future endogenous response.

Thus the raw implementation state has:

$$48 + 16 = 64$$

dimensions.

This cache is behaviorally null by construction.

For each benchmark run, the entire 64-dimensional snapshot is transformed by a new random orthogonal matrix. The MECS learner therefore never receives a privileged coordinate system separating recurrent memory from the null register.

S14.3 Endogenous future-response signature

For each frozen recurrent snapshot, we evaluate the GRU under five standardized one-step probe inputs:

1. no-op;
2. flip latent-control channel 1;
3. flip channel 2;
4. flip channel 3;
5. flip channel 4.

No new sensory bit is revealed during the probe, and nuisance inputs are set to zero.

The concatenated future readouts form a 20-dimensional response signature.

The full-probe MECS learner is an eight-dimensional bottleneck trained only to predict this response signature from the arbitrarily recoded raw implementation state.

It is **never** trained on:

- the true latent environment state;
- the frozen GRU’s current 16-class belief/readout label;
- the identity of the null cache coordinates.

The belief class and world state are used only for evaluation.

The belief-class probe is nevertheless **not statistically independent of the training target**: both are readouts of the same frozen recurrent system. Its decoding accuracy should therefore be interpreted as a consistency diagnostic showing that the learned bottleneck preserves distinctions expressed in the system’s own response family, not as an independent external validation of the quotient.

S14.4 Baselines

We compare:

- **MECS-Full5Probe-8**: eight-dimensional predictive bottleneck trained on the complete five-probe response signature;
- **MECS-Limited2Probe-8**: same architecture trained on only two probe responses;
- **Autoencoder-8**: eight-dimensional reconstruction bottleneck;
- **PCA-8**: first eight principal components of the raw implementation state.

The experiment is repeated under five independent orthogonal recodings.

S14.5 Results

Chance accuracy for a 16-class functional probe is:

$$1/16 = 0.0625.$$

Representation	Split	Agent-belief class accuracy	True environment- state accuracy	Null-cache R^2	Within-belief variance ratio	Signature MSE
MECS-Full5Probe-8	ID	0.955±0.003	0.199±0.002	0.2007±0.0004	0.146±0.005	0.0036±0.0006
MECS-Full5Probe-8	Nuisance/cache	0.927±0.004	0.196±0.001	—	0.171±0.005	0.0059±0.0009
MECS-Limited2Probe-8	ID	0.949±0.002	0.198±0.001	0.1018±0.0004	0.147±0.009	0.0033±0.0006

Representation	Split	Agent-belief	True environment-state	Null-cache	Within-belief	Signature MSE
		class accuracy	accuracy	R^2	variance ratio	
Autoencoder-8	ID	0.064±0.000	0.0362±0.000	0.02484±0.000	0.01997±0.000	—
PCA-8	ID	0.063±0.000	0.0362±0.000	0.0501±0.000	0.01997±0.000	—

S14.6 Functional state versus world state

The full-probe representation decodes the frozen recurrent system’s current 16-class **belief state** with approximately:

$$95.5$$

accuracy.

Its accuracy for the true external 16-state world is only:

$$19.9$$

This difference is not interpreted as a failed quotient.

The recurrent system is partially observable and itself achieves only imperfect bitwise world-state tracking. MECS is defined by the recurrent process’s future endogenous behavior, not by access to an omniscient simulator state.

The benchmark therefore empirically illustrates Corollary 3:

$$\text{agent-relative functional state} \neq \text{external world state}.$$

S14.7 Null-cache rejection

The behaviorally null cache is highly represented by reconstruction-oriented baselines:

$$R_{\text{cache}}^2 \approx 0.48\text{-}0.50.$$

For the full-probe predictive bottleneck:

$$R_{\text{cache}}^2 \approx 0.007.$$

At the same time, the autoencoder and PCA remain near the 16-class chance level for the agent-belief state:

$$\approx 0.063.$$

Thus the bottleneck uses its limited capacity to preserve recurrent distinctions that matter to the frozen system’s own future response and rejects implementation state that is behaviorally inert.

S14.8 Nuisance/cache distribution shift

The observation nuisance and implementation cache are shifted far outside their training mean while the frozen recurrent process and functional dynamics remain unchanged.

The full-probe MECS representation retains belief-class accuracy:

$$92.7$$

with response-signature MSE:

$$0.0059.$$

The out-of-distribution linear cache R^2 is not interpreted quantitatively because R^2 under strong mean shift is not an information measure.

S14.9 Probe-family coverage

The two-probe model reaches:

$$94.9$$

belief-class accuracy, only modestly below the five-probe model.

This benchmark therefore does **not** show a strong intervention-coverage requirement.

Instead it shows that, for this trained GRU, two standardized probes already contain most of the response information needed to identify the recurrent functional state.

This is consistent with Proposition 3: identifiability depends on the distinctions induced by the available intervention family, not merely on the number of interventions.

S14.10 Methodological relation to hidden-state intervention work

The recurrent benchmark intentionally does not write arbitrary vectors into the GRU hidden state.

All standardized probes enter through the same action/observation interface used during training. This reduces, but does not eliminate, the representation-divergence concern raised by Grant et al. (2026), who show that hidden-state interventions can create unnatural internal states.

Likewise, the benchmark does not claim that the recovered eight-dimensional code is one fixed geometric subspace of the GRU hidden dynamics. Recent recurrent-world-model work shows that intervention-relevant low-rank structure can move with state through high-dimensional dynamics.

The code here is a learned quotient representation for a declared family of future responses, not a claim about a fixed physical subspace inside the GRU.

S14.11 Remaining limitations

This recurrent benchmark is materially stronger than the static synthetic generator, but it remains controlled.

- The recurrent system is trained by supervised latent-state tracking rather than reward-maximizing reinforcement learning.
- The exogenous action family is small.
- The persistent cache is synthetically appended and exactly behaviorally null.
- The MECS learner is retrained for each arbitrary raw-state recoding rather than required to generalize zero-shot across recodings.
- The full five-probe response signature is computed from one-step probes rather than long-horizon future trajectories.

The next benchmark should use a persistent reward-driven agent with naturally emerging long-horizon memory, then estimate the quotient and $\Gamma(S)$ across actual fork, restore, and merge events.

S15. Reward-driven persistent-agent fork, restore, and merge benchmark

The previous recurrent benchmark used a policy trained for latent-state tracking. We now train a separate recurrent policy **only through delayed task reward**.

S15.1 Reward-driven recurrent process

The environment contains a stable four-bit hidden state. At each observation step, the policy sees:

- one randomly selected hidden bit;
- 4% observation noise;
- four unrelated nuisance features.

The GRU has:

48

hidden units.

The policy outputs four Bernoulli decisions corresponding to a four-bit guess. Reward is the final fraction of correctly guessed bits after a 14-step partially observed sequence.

The training objective is the exact expected delayed task reward under the Bernoulli policy:

$$J(\pi) = E_{\pi} \left[\frac{1}{4} \sum_{j=1}^4 \mathbf{1}(A_j = Z_j) \right].$$

No latent-state classification loss, hidden-state reconstruction loss, or MECS loss is used.

After 2,200 reward updates:

- mean bit reward, ID:

0.868

- exact four-bit state rate, ID:

0.552

- mean bit reward under a large nuisance shift:

0.837.

The agent is then frozen.

S15.2 Natural-interface intervention payload

To probe lineage, a balanced four-bit intervention variable:

$$U \in \{0, \dots, 15\}$$

is delivered through the policy’s ordinary observation interface.

Each payload bit is presented exactly once as a noiseless observation. This creates a post-payload live recurrent state.

Fork and restore operations are then applied at different causal times.

The future response of each branch is measured by the frozen policy’s next null-observation action.

Inheritance is summarized as:

$$\Gamma(B) = \frac{I(U; A_B)}{H(U)},$$

and similarly for other processes and future sets.

Because:

$$H(U) = 4 \text{ bits},$$

the reported values lie in $[0, 1]$.

S15.3 Actual fork timing

Process event	$\Gamma(B)$	$\Gamma(C)$	$\Gamma(B, C)$	Pair interaction
Ordinary live continuation	0.781	0.000	0.781	0.000
Full fork after payload	0.781	0.781	0.781	-0.781
Fork before payload; update B only	0.781	0.000	0.781	0.000

Process event	$\Gamma(B)$	$\Gamma(C)$	$\Gamma(B, C)$	Pair interaction
Stale restore replaces live state	0.000	—	—	—

The timing distinction is exact.

When the fork occurs **after** the payload, both descendants retain the same post-intervention information. The negative interaction term is the expected redundancy signature of duplicated recurrent state.

When the fork occurs **before** the payload and only branch B receives the intervention:

$$\Gamma(B) = 0.781,$$

$$\Gamma(C) = 0.000.$$

A stale restore to the pre-payload checkpoint produces:

$$\Gamma = 0.000.$$

Thus the inherited payload follows process lineage rather than process naming.

S15.4 Redundant versus complementary distributed continuation

Two branches are created from the same pre-payload checkpoint.

For the redundant-half condition, both branches receive payload bits 1-2.

For the complementary condition:

- branch B receives bits 1-2;
- branch C receives bits 3-4.

Condition	$\Gamma(B)$	$\Gamma(C)$	$\Gamma(B, C)$	Interaction
Redundant half-fork	0.500	0.500	0.500	-0.500

Condition	$\Gamma(B)$	$\Gamma(C)$	$\Gamma(B, C)$	Interaction
Complementary fork	0.500	0.500	1.000	0.000

The singleton inheritance scores are identical:

$$\Gamma(B) = \Gamma(C) = 0.5.$$

Yet:

$$\Gamma(B, C) = 0.5$$

under redundancy and:

$$\Gamma(B, C) = 1$$

under complementarity.

This reproduces the paper’s set-valued inheritance distinction inside a reward-trained recurrent policy using only the normal input/output interface.

S15.5 Learned merge

A separate merge operator receives the two complementary branch hidden states and outputs one new 48-dimensional recurrent state.

The merger is trained **only to maximize the frozen policy’s downstream task reward**.

It is not supervised with the four-bit payload label.

Before merge, the two complementary branches each achieve roughly 0.75 bit-wise task reward.

After merge, the merged process reaches:

$$100\%$$

bit-wise reward and:

$$100\%$$

exact-state accuracy in the held-out merge evaluation.

Its inherited payload information is:

$$\Gamma(M) = 1.000.$$

Thus two partial recurrent processes can be recombined into a single process whose downstream task behavior recovers the full distributed payload.

S15.6 Detached reconstruction exposes a causal confound

A fresh recurrent process can be rebuilt from the same four-bit payload after the live post-payload process has been discarded.

The reconstructed process yields:

$$\Gamma_{\text{upstream}} = 0.781$$

when inheritance is naively measured against the original payload U .

This is approximately the same as the live continuation.

Yet the detached reconstruction need not be causally descended from the **post-payload live recurrent state**.

This is not a failure of the benchmark. It is the empirical counterexample motivating Proposition 4.

The payload is a common upstream cause of both:

- the live process;
- the reconstruction.

Therefore:

$$I(U; Y)$$

alone does not identify current-process lineage.

To test descent from the current process, one must intervene **after** the reconstruction path has become causally separated, or intervene directly on the current MECS state.

S15.7 Refining the inheritance notation

When Γ is intended to mean **current-process inheritance**, the intervention source should be explicit:

$$\Gamma_{A_t \rightarrow S}^{\mathcal{J}}$$

rather than simply:

$$\Gamma(S).$$

Here:

- A_t identifies the current process being intervened upon;
- $\mathcal{J}$ identifies the admissible post-divergence intervention family;
- S identifies the future process set.

The shorter notation $\Gamma(S)$ remains useful when the intervention source is unambiguous.

S15.8 What this benchmark establishes

The experiment shows, in one reward-driven recurrent process, that:

1. full post-intervention fork duplicates inherited information;
2. pre-intervention branch timing determines which descendants inherit the perturbation;
3. stale restore can erase post-checkpoint functional information;
4. redundant and complementary branch pairs can have identical singleton inheritance but different joint inheritance;
5. a learned merge can reconstitute distributed functional information into one recurrent process;
6. upstream mutual information can misidentify detached reconstruction as continuation unless the current process is the intervention target.

S15.9 Full-stack replication across independently trained recurrent processes

The core lineage benchmark is repeated with three independently initialized and trained reward-driven GRUs. A separate merger network is trained for each GRU.

The independently trained agents differ substantially in task proficiency:

Stack seed	Mean bit reward	Exact-state rate
20263001	0.6976	0.1420
20263002	0.8638	0.5095
20263003	0.8389	0.4540

Despite this performance variation, the set-level inheritance structure is consistent across all three stacks.

Scenario	$\Gamma(B)$	$\Gamma(C)$	$\Gamma(B, C)$	$\Gamma(M)$	Pair interaction
complementary	0.359 ± 0.149	0.417 ± 0.072	0.776 ± 0.089	—	0.000 ± 0.000
full_fork	0.603 ± 0.204	0.603 ± 0.204	0.603 ± 0.204	—	-
merged_complementary	0.359 ± 0.149	0.417 ± 0.072	0.776 ± 0.089	0.875 ± 0.000	0.603 ± 0.204
ordinary	0.603 ± 0.204	0.000 ± 0.000	0.603 ± 0.204	—	0.000 ± 0.000
redundant	0.359 ± 0.149	0.359 ± 0.149	0.359 ± 0.149	—	-
stale_restore	0.000 ± 0.000	—	—	—	0.359 ± 0.149

All three stacks satisfy the following structural checks:

- full-fork singleton inheritance equals joint inheritance, producing a negative redundancy interaction;
- redundant half-forks have $\Gamma(B) = \Gamma(C) = \Gamma(B, C)$;
- complementary forks satisfy $\Gamma(B, C) = \Gamma(B) + \Gamma(C)$ at the measured policy readout;
- stale restore carries zero information about the post-checkpoint payload;
- the learned merger recovers at least the measured joint complementary inheritance in every tested stack.

Absolute Γ values vary because the downstream readout is task-dependent and the independently trained agents differ in proficiency. The replicated claim is therefore structural, not that a particular numeric inheritance magnitude is universal.

This experiment is a full-stack replication of the **core lineage constructions**. It is not a full-stack replication of the end-to-end long-horizon process-aware versus context-only controller comparison.

S15.10 Limitations

The experiment remains deliberately small.

- The agent’s hidden state has only 48 dimensions.
- The hidden world contains four stable bits.
- The “fork” operation copies recurrent hidden state exactly.
- The merge operator is an auxiliary learned map rather than an endogenous merge operation discovered by the policy.
- The current-process intervention is implemented through a natural observation payload rather than a direct MECS manipulation.

- The benchmark does not yet include long-horizon autonomous goals across fork and merge.

The result is therefore a functional lineage stress test, not an empirical model of human identity or a complete persistent-agent architecture.

S16. Direct post-separation current-process intervention test

Supplementary Note S15 showed that a detached reconstruction generated from the same upstream payload can retain high mutual information with that payload. To distinguish common ancestry from continuation of the **current** process, we run a second lineage test.

S16.1 Protocol

A neutral base recurrent state is first established.

The live process and a detached/stale path are then causally separated.

Only after this separation, a new balanced four-bit intervention variable:

$$J \in \{0, \dots, 15\}$$

is delivered to the live process through the same ordinary observation interface used during training.

The detached path never receives J .

We then evaluate the next policy readout of each candidate process.

S16.2 Results

Scenario	$\Gamma_J(B)$	$\Gamma_J(C)$	$\Gamma_J(B, C)$
Live continuation after J	0.373	—	—
Detached pre- J reconstruction	0.000	—	—
Full fork after J	0.373	0.373	0.373
Fork before J ; update B only	0.373	0.000	0.373
Stale restore to pre- J checkpoint	0.000	—	—

The direct separation is now unambiguous.

Live continuation

$$\Gamma_J = 0.373.$$

Detached reconstruction

$$\Gamma_J = 0.000.$$

The detached path shares the entire pre- J process history but contains no information about the post-separation intervention.

Fork after intervention

Both descendants inherit the current-process perturbation:

$$\Gamma_J(B) = \Gamma_J(C) = 0.373.$$

Joint readout inheritance remains:

$$\Gamma_J(B, C) = 0.373,$$

showing complete redundancy at the measured policy interface.

Fork before intervention

Only the updated branch carries the intervention:

$$\Gamma_J(B) = 0.373,$$

$$\Gamma_J(C) = 0.000.$$

Stale restore

Restoring the pre- J checkpoint produces:

$$\Gamma_J = 0.000.$$

S16.3 What this establishes

The experiment demonstrates, within the same reward-trained recurrent process, that:

shared pre-divergence history $\nRightarrow$ current-process inheritance.

The same detached/stale process that appears continuous with respect to older shared information becomes information-theoretically disconnected when the intervention is moved to the current process after divergence.

S16.4 Readout-level lower bound

The measured value:

$$0.373$$

is computed from the policy’s next four-bit readout.

By Corollary 4:

$$\Gamma_J^{\text{readout}} \leq \Gamma_J^{\text{hidden}}.$$

The benchmark therefore establishes a positive lower bound on live current-process inheritance at the policy interface; it does not claim that the recurrent hidden state retains only 37.3% of the current process’s total functional information.

S17. Autonomous persistence-operation selection

The preceding benchmarks evaluate lineage and preservation **after** operations are specified. We next allow a learned controller to choose the persistence operation itself.

The frozen reward-driven recurrent process from Supplementary Note S15 is reused.

S17.1 Operation set

At each persistence decision, the controller selects one of four actions:

1. **KEEP** — continue the live post-update process;
2. **FORK** — create two redundant full copies of the live process;

3. **RESTORE** — revert to the stale pre-update checkpoint;
4. **SPLIT+MERGE** — create complementary branches from the checkpoint and use the learned merger if both survive.

The environment supplies:

- a noisy confidence cue indicating whether the current update is task-relevant;
- independent branch failure probability;
- fork cost;
- restore cost;
- split cost;
- merge cost.

The recurrent agent’s own predicted bit probabilities for candidate live, stale, branch-B, branch-C, and merged states are available as prospective internal planning signals.

The controller does **not** receive a symbolic instruction such as “fork now” or “restore now.”

S17.2 Expected task utilities

Let r_L , r_O , r_B , r_C , and r_M denote the frozen recurrent agent’s actual bit-wise task rewards if the corresponding live, old, branch, or merged state is ultimately used.

Let p be independent branch failure probability and c , operation costs.

The synthetic benchmark uses:

$$U_{\text{KEEP}} = (1 - p)r_L + p r_0,$$

$$U_{\text{FORK}} = (1 - p^2)r_L + p^2 r_0 - c_F,$$

$$U_{\text{RESTORE}} = (1 - p_R)r_O + p_R r_0 - c_R,$$

and:

$$U_{\text{SPLIT}} = (1 - p)^2(r_M - c_M) + p(1 - p)r_B + p(1 - p)r_C + p^2 r_0 - c_S,$$

where:

$$r_0 = 0.5$$

is the default bit reward if no usable process survives.

These utilities are benchmark evaluation functions, not metaphysical survival values.

S17.3 Full-information entropy-regularized controller

A controller is first trained against the entropy-regularized reward-optimal action distribution:

$$\pi^*(a \mid x) \propto \exp(\beta U(a, x)),$$

with:

$$\beta = 25.$$

The controller receives no discrete operation label; the target distribution is derived from counterfactual task reward.

On 20,000 held-out episodes:

$$\text{mean regret} = 0.0043,$$

$$\text{oracle action match} = 88.9\%.$$

For comparison, always KEEP has mean regret 0.1128 and a myopic KEEP-versus-RESTORE rule based only on update confidence has mean regret 0.0528.

The learned deterministic policy selects all four persistence operations:

- KEEP: 25.3%;
- FORK: 10.7%;
- RESTORE: 40.4%;
- SPLIT+MERGE: 23.6%.

S17.4 Same-distribution information ablation

All controllers below use the same architecture family, reward-derived policy target, train/test episodes, and optimization schedule. Only the available planning information changes.

Planning information	Mean reward	Mean regret	Zero-regret rate	Oracle-action match
Context only	0.7221	0.0464	0.590	0.590
Context + live/stale predictions	0.7601	0.0084	0.838	0.838
Full counter-factual branch/merge predictions	0.7639	0.0045	0.887	0.887

The context-only controller cannot reliably infer when the recurrent implementation makes FORK or SPLIT+MERGE worthwhile.

Adding live and stale process predictions produces a large improvement:

$$0.0464 \rightarrow 0.0084$$

mean regret.

Adding explicit branch-B, branch-C, and merged-state counterfactual predictions produces a further reduction to approximately:

$$0.0045.$$

Thus the benefit is not explained solely by failure probability and operation costs; prospective process-state information materially improves persistence control.

S17.5 Bandit-feedback actor-critic

The full-information controller still sees all counterfactual operation rewards during training. We therefore train a second controller using contextual-bandit feedback.

At each training update:

1. the actor samples exactly one persistence action;
2. only the sampled action’s task utility is revealed;
3. a learned critic provides the baseline;
4. unchosen action utilities are not used in the policy-gradient update.

On the same 20,000-episode test distribution:

$$\text{mean regret} = 0.0086,$$

$$\text{zero-regret rate} = 81.9\%,$$

$$\text{oracle action match} = 81.9\%.$$

The controller learns all four persistence operations:

Oracle-optimal operation	Test count	Controller match	Mean regret
KEEP	5318	0.778	0.0095
FORK	2024	0.602	0.0125
RESTORE	7820	0.886	0.0065
SPLIT+MERGE	4838	0.846	0.0094

FORK remains the hardest action to learn under bandit feedback, but its learned use is nonzero and substantially above the collapse observed in the first context-only controller.

S17.6 Multi-seed contextual-bandit replication

The contextual-bandit controller is retrained from six independent parameter initializations and stochastic action trajectories on the same fixed train/test distribution.

Across the six training seeds:

$$\text{mean regret} = 0.00851 \pm 0.00060$$

with seed-level 95% t interval:

$$[0.00788, 0.00914],$$

and mean oracle-action agreement:

$$81.6$$

with 95% interval:

$$[80.3$$

Every seed assigns nonzero test-time action share to all four persistence-operation classes. The mean action shares are:

- KEEP: 26.0%;
- FORK: 9.6%;
- RESTORE: 39.8%;
- SPLIT+MERGE: 24.6%.

The always-KEEP reference regret on the same test set is:

$$0.1128.$$

This replication supports the narrow claim that the persistence-operation policy is learnable from chosen-action reward feedback in this controlled contextual task.

The train and test episode sets are held fixed across seeds, so the reported uncertainty concerns controller optimization/initialization, not distributional uncertainty over possible task generators.

S17.7 What this establishes

The benchmark shows that a persistent recurrent system can learn a **task-dependent persistence policy** over branch, restore, and merge operations.

It also shows a hierarchy of planning information:

context < live/stale process predictions < full branch/merge counterfactual predictions.

This is consistent with the paper’s broader claim that persistence decisions depend not only on which processes are successors, but on the functional structure available in candidate future process sets.

S17.8 What this does not establish

The benchmark remains a contextual persistence-control task rather than a full autonomous life-history MDP.

- Operation semantics are defined by the environment.
- Branch/failure outcomes occur within one persistence decision.
- The controller does not invent new persistence operations.
- The agent does not reason over legal, social, or phenomenal identity.
- The full-information controller uses counterfactual reward access unavailable in many real systems.
- Even the bandit controller uses a fixed synthetic distribution and a learned recurrent process with a small state space.

A stronger future benchmark should make persistence operations temporally extended actions whose consequences affect future learning, future branch opportunities, and long-horizon goals.

S18. Long-horizon sequential persistence MDP

The autonomous-control benchmark in Supplementary Note S17 contains one persistence decision. We now construct a finite-horizon MDP in which persistence operations change later states and future operation values.

S18.1 State and action space

The MDP state is:

$$s_t = (q_t, c_t, m_t, h_t, g_t),$$

where:

- $q_t \in \{0, \dots, 4\}$: current task-relevant functional quality;
- $c_t \in \{0, \dots, 4\}$: checkpoint quality;
- $m_t = 0$: one live process;
- $m_t = 1$: redundant fork;
- $m_t = 2$: complementary split;
- $h_t \in \{0, 1\}$: low/high failure-risk regime;
- $g_t \in \{0, 1\}$: low/high innovation regime.

This yields:

$$5 \times 5 \times 3 \times 2 \times 2 = 300$$

states.

The action set is:

$\{KEEP, CHECKPOINT, FORK, RESTORE, SPLIT, MERGE\}$.

S18.2 Transition semantics

FORK.

A redundant fork changes catastrophic failure probability from p to:

$$p^2,$$

but incurs creation and maintenance cost.

SPLIT.

A complementary split lowers immediate usable reward because no single branch contains the full process state, but under high innovation it has a higher probability of improving joint potential quality.

MERGE.

MERGE converts the current distributed potential back into one executable process.

CHECKPOINT.

A checkpoint stores current quality but becomes stale with nonzero probability as the task/process environment evolves.

RESTORE.

Restore replaces live quality with the checkpoint and collapses branch state.

The failure-risk and innovation regimes evolve as two-state Markov processes.

S18.3 Exact finite-horizon oracle

The horizon is:

$$T = 12$$

with discount:

$$\gamma = 0.97.$$

Because the transition model is finite and enumerable, exact backward dynamic programming provides a reference optimum.

Over 30,000 initial states sampled from the benchmark’s evaluation distribution:

Policy / action restriction	Mean discounted return	Regret vs full oracle
Exact full-action DP oracle	7.7572	0
Naive online tabular Q-learning	7.1414	0.6158
Myopic one-step policy	6.8566	0.9005
Exact optimum without SPLIT/MERGE	7.7111	0.0460
Exact optimum without any branch operation	7.0091	0.7481

Three quantitative conclusions follow.

First, myopic persistence control is substantially suboptimal:

$$\text{myopic regret} = 0.9005.$$

Second, removing all branch-producing persistence operations causes:

$$\Delta_{\text{branch}} \approx 0.7481$$

mean discounted-value loss under the evaluation distribution.

Third, SPLIT/MERGE has positive but comparatively small option value in this parameterization:

$$\Delta_{\text{split}} \approx 0.0460.$$

This calibration matters. The benchmark does not force distributed continuation to dominate every task. It shows a regime in which

redundancy/forking is a major long-horizon option and split/merge is a smaller but measurable additional option.

S18.4 Oracle operation distribution

At time $t = 0$, across the complete 300-state space, the full DP oracle uses every persistence operation:

Operation	Oracle state share
KEEP	0.080
CHECKPOINT	0.337
FORK	0.053
RESTORE	0.443
SPLIT	0.040
MERGE	0.047

The long-horizon optimum therefore cannot be reduced to one dominant edit rule.

S18.5 Negative result: naive online Q-learning

The model-free tabular Q-learning baseline receives only sampled transitions and rewards.

After:

90,000

training episodes, its mean regret is still:

0.6158,

and global state-time oracle-action agreement is only:

0.376.

The learned $t = 0$ policy collapses strongly toward KEEP.

This result is retained as a negative result.

It shows that the existence of a compact finite MDP does not imply that naive online Q-learning efficiently discovers rare option-value operations under the tested exploration and learning-rate schedule.

S18.6 Sample-based model learning

To separate persistence-planning value from model-free learning difficulty, a second learner receives only sampled next-state/reward observations for each admissible state-action pair.

It estimates an empirical transition model and then performs finite-horizon planning in that learned model.

No oracle actions or exact transition probabilities are provided.

Samples per state-action	Mean return	Regret vs oracle	Oracle-action agreement
10	7.2377±0.1187	0.5261±0.1187	0.718±0.024
25	7.4652±0.0647	0.2986±0.0647	0.793±0.009
50	7.5931±0.0321	0.1707±0.0321	0.828±0.015
100	7.6643±0.0287	0.0995±0.0287	0.861±0.011
300	7.7152±0.0119	0.0486±0.0119	0.908±0.007

Regret decreases monotonically over the tested data budgets.

At:

$$300$$

samples per state-action pair, the learned-model planner reaches:

$$\text{regret} = 0.0486$$

and:

$$\text{oracle-action agreement} = 90.8.$$

This is close to the full-action exact optimum and much better than the naive online Q-learning schedule.

S18.7 Interpretation

The long-horizon benchmark supports three distinct claims.

1. Planning depth matters.

One-step persistence control leaves substantial value on the table.

2. **Persistence action expressivity matters.**

An architecture that forbids branching loses substantial option value even under optimal planning.

3. **Learning algorithm matters separately.**

Model-free online Q-learning can fail to discover those options efficiently even though sample-based model learning and planning recover most of the oracle value.

These claims should not be conflated.

S18.8 Relation to BPO and persistence systems

Branching Policy Optimization uses checkpoint/fork operations to improve the *training estimator* for language agents. The present MDP instead places persistence operations **inside the agent’s decision problem**.

The Transactional Continuity Kernel governs which proposed branch head is authorized and fresh enough to become active. The present MDP assumes admissible operations and studies their future task value.

Checkpoint-handoff evaluation separates reaching a state from solving from that state. The present benchmark similarly relies on state replayability but asks how the choice of persistence edit changes later state distributions and later option value.

S18.9 Limitations

The MDP is still abstract.

- Functional quality is compressed to five discrete levels.
- Failure and innovation regimes are hand-designed Markov processes.
- The branch modes summarize redundancy/complementarity rather than simulating full recurrent hidden-state branches.
- The exact oracle knows the benchmark transition model.
- The sample-based learner observes every admissible state-action pair by design.

A stronger end-to-end experiment should expose the reward-driven recurrent agent from Supplementary Notes S15–S17 to a multi-step persistence environment and require it to learn both functional continuation models and persistence policy from its own trajectories.

S19. End-to-end recurrent long-horizon persistence integration

Supplementary Notes S15–S17 manipulate the reward-trained recurrent process directly but either measure lineage or optimize a single persistence decision. Supplementary Note S18 introduces sequential persistence value in an abstract finite MDP. We now combine these two directions.

The frozen reward-trained GRU hidden state itself is carried across repeated persistence decisions.

S19.1 Hidden-state persistence mechanics

At each macro-step, persistence operations act directly on recurrent state:

- **CHECKPOINT** copies the current GRU hidden state into a persistent checkpoint;
- **FORK** clones the current hidden state into two recurrent branches;
- **RESTORE** replaces the live hidden state with the stored checkpoint;
- **SPLIT** clones the current hidden state into two branches that subsequently receive complementary observations;
- **MERGE** applies the learned recurrent merger to the two branch hidden states.

The latent four-bit world can change between persistence decisions.

Low/high risk regimes change branch-failure probability. Low/high innovation regimes change the probability that the world itself changes. A checkpoint can therefore become functionally stale without its stored bytes changing.

Branch failures act on the actual recurrent states. If one redundant or complementary branch fails, the surviving recurrent state becomes the live single process. If both fail, the recurrent state is reset.

The controller receives persistence context and, in the process-aware condition, the frozen recurrent agent’s predicted outputs from:

- the live recurrent state;
- the checkpoint;
- branch B;
- branch C;
- the prospective merged state.

S19.2 State-sensitive calibration

The first environment uses moderate failure risk and makes complementary split useful for gathering distributed observations.

All policies are evaluated on the same stochastic environment seeds.

This table reports the original single-training-seed pilot. It is retained for transparency, but the multi-seed replication in Supplementary Note S19.3 is the primary inferential result and supersedes any interpretation of the single-seed return difference.

Policy	turn	Mean dis- counted re-						Fork- mode share	Split- mode share
		KEEP	CHECK	FORK	TRESTOP	SPLIT	MERGE	share	share
Process-aware long-horizon	50298	0.794	0.000	0.000	0.000	0.206	0.000	0.000	0.316
Context-only long-horizon	418946	1.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Process-aware, no branch actions	457901	0.000	1.000	0.000	0.000	0.000	0.000	0.000	0.000
KEEP only	4.8946	1.000	0	0	0	0	0	0	0

In this single training run, the process-aware controller improves over context-only by:

0.1352

discounted return and learns a KEEP/SPLIT strategy.

This difference is retained as an **existence-style optimization outcome only**. It is not used as evidence for a reliable process-aware advantage because the multi-seed replication below does not confirm it.

S19.3 Controller-level multi-seed replication of the state-sensitive comparison

The process-aware and context-only long-horizon controllers are retrained across eight independent controller-training seeds while holding the underlying reward-trained GRU and merger fixed.

Each trained controller is evaluated on the same 3,000 stochastic environment episodes.

The seed-level results are:

$$\bar{V}_{\text{process}} = 5.0265,$$

$$\bar{V}_{\text{context}} = 5.0275,$$

with paired mean difference:

$$\Delta\bar{V} = -0.0011.$$

The 95% t interval is:

$$[-0.2904, 0.2883],$$

and a nonparametric bootstrap over training seeds gives:

$$[-0.2239, 0.2211].$$

The paired t-test gives:

$$p = 0.993.$$

Four of eight seeds favor the process-aware controller, three favor context-only, and one is effectively tied under the numerical threshold used in the analysis.

Therefore the multi-seed replication does **not** establish a process-aware return advantage.

This is the primary inferential result for the state-sensitive end-to-end comparison. The earlier single-seed positive run should be read only as evidence that a process-aware controller can sometimes discover a useful split strategy, not that it does so reliably.

The replication holds the underlying recurrent agent and merger fixed. A full-stack replication across independently trained recurrent agents remains unperformed.

S19.4 Risk-dominant calibration: a deliberate null result

A second calibration increases high-risk branch failure pressure, randomizes persistence-operation costs, and makes redundant FORK comparatively valuable.

Policy	Mean dis- counted re- turn	KEEP	CHECKPOINT	FORK	RESTORE	SPLIT	MERGE	Fork- mode share
Process-aware long-horizon	6.1126	0.627	0.000	0.373	0.000	0.000	0.000	0.696
Context-only long-horizon	6.1126	0.627	0.000	0.373	0.000	0.000	0.000	0.696
Process-aware, no branch actions	5.7602	0.818	0.182	0.000	0.000	0.000	0.000	0.000

The process-aware and context-only controllers converge to the same policy:

$$P(KEEP) \approx 0.627,$$

$$P(FORK) \approx 0.373.$$

They obtain identical return:

$$6.1126.$$

The no-branch controller remains lower:

5.7602.

This is an important null result.

Detailed recurrent-process predictions are **not universally useful**.

When failure risk and operation cost already identify the dominant persistence decision, a more detailed inheritance-aware representation should not be expected to improve control.

S19.5 Revised interpretation across calibrations and seeds

The combined evidence no longer supports a stable end-to-end return advantage for process-aware features.

What is supported is weaker:

1. individual optimization runs can discover process-aware split strategies;
2. simpler context can be sufficient in risk-dominant regimes;
3. across eight controller seeds in the state-sensitive calibration, process-aware and context-only mean return are statistically indistinguishable at the seed level.

Accordingly, the paper does **not** claim that richer successor-process features reliably improve long-horizon recurrent control. Their decision value remains an open empirical question requiring better training stability, full-stack replication, and richer environments.

S19.6 Remaining integration gap

The end-to-end benchmark is still incomplete in one important sense.

No single learned recurrent long-horizon policy in the tested calibrations robustly uses the full operation set.

The state-sensitive policy uses SPLIT but does not learn MERGE. The risk-dominant policy uses FORK but does not use SPLIT/MERGE. CHECKPOINT and RESTORE remain more prominent in the one-decision and abstract-MDP experiments than in the integrated recurrent controller.

Therefore the evidence should be interpreted compositionally:

- reward-trained recurrent lineage experiments establish fork/restore/merge semantics;
- autonomous contextual control establishes learned selection among all four major operation families;

- the abstract MDP establishes long-horizon option value for all persistence operations;
- the end-to-end recurrent experiment establishes that actual recurrent states can participate in multi-step branch-aware control, but does not yet unify the full operation set in one learned policy.

S20. Joint successor modeling does not require explicit 2^n enumeration

A conceptual successor-set posterior is a distribution over:

$$R \subseteq \mathcal{C}.$$

This does not require implementation as one flat softmax with 2^n logits.

Let:

$$X_i = \mathbf{1}(c_i \in R).$$

Any joint distribution over the successor set can be factorized by the chain rule:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid X_{<i}).$$

Thus the representational requirement is **dependency capacity**, not a particular exponential table.

Possible scalable parameterizations include:

- autoregressive inclusion models;
- factor graphs;
- energy-based set models;
- exchangeable random-set models where justified;
- sparse higher-order interaction models;
- structured variational approximations.

These parameterizations impose different inductive biases. The theory requires that the model not be forced to destroy decision-relevant successor dependence.

Appendix A. Exact Analytical Benchmark

Let successor indicators be:

$$X = (B, C) \in \{00, 10, 01, 11\}.$$

Scenario	$P(00)$	$P(10)$	$P(01)$	$P(11)$	TC
Unitary-B	0	1	0	0	0
Full-fission	0	0	0	1	0
Exclusive uncertainty	0	0.5	0.5	0	1 bit
Fission-or-death	0.5	0	0	0.5	1 bit
Independent 0.5	0.25	0.25	0.25	0.25	0

The best independent model is the product of marginals.

For both correlated matched-marginal scenarios:

$$q(B = 1) = q(C = 1) = 0.5,$$

so the independent model predicts all four states with probability 0.25.

True entropy:

$$H(X) = 1 \text{ bit.}$$

Independent cross entropy:

$$H_{\text{indep}} = 2 \text{ bits.}$$

Excess NLL:

$$1 \text{ bit.}$$

Appendix B. Möbius Signatures

For two future candidates:

$$m_{BC} = \Gamma(BC) - \Gamma(B) - \Gamma(C).$$

Scenario	m_{BC}
Full duplicate	-1
Half redundant copy	-0.5
Additive complementary split	0
Secret-share synergy	+1

These are exact values for the toy constructions, not empirical estimates.

References

1. Gallagher, S. (2013). A Pattern Theory of Self. *Frontiers in Human Neuroscience*, 7, 443. <https://doi.org/10.3389/fnhum.2013.00443>
2. Apps, M. A. J., & Tsakiris, M. (2014). The free-energy self: A predictive coding account of self-recognition. *Neuroscience & Biobehavioral Reviews*, 41, 85-97. <https://doi.org/10.1016/j.neubiorev.2013.01.029>
3. Limanowski, J., & Blankenburg, F. (2013). Minimal self-models and the free energy principle. *Frontiers in Human Neuroscience*, 7, 547. <https://doi.org/10.3389/fnhum.2013.00547>
4. Kirchhoff, M., Parr, T., Palacios, E., Friston, K., & Kiverstein, J. (2018). The Markov blankets of life: autonomy, active inference and the free energy principle. *Journal of the Royal Society Interface*, 15(138), 20170792. <https://doi.org/10.1098/rsif.2017.0792>
5. Liang, C., Lin, W.-H., Liou, W.-K., Chen, B.-Y., Lin, J.-R., Lee, Y.-T., & Chen, S. (2025). Double body effect induced by integrating proprioceptive-vestibular and visual information. *iScience*, 28(11), 113819. <https://doi.org/10.1016/j.isci.2025.113819>
6. Olson, E. T. Personal Identity. *Stanford Encyclopedia of Philosophy*. Current online edition: <https://plato.stanford.edu/entries/identity-personal/>
7. Ehring, D. (2021). *What Matters in Survival: Personal Identity and Other Possibilities*. Oxford University Press. <https://doi.org/10.1093/oso/9780192894717.001.0001>
8. Otsuka, M. (2018). Personal Identity, Substantial Change, and the Significance of Becoming. *Erkenntnis*, 83, 1229-1243. <https://doi.org/10.1007/s10670-017-9938-7>

9. Cox, R. (2026). De Se pluralism. *Inquiry*. <https://doi.org/10.1080/0020174X.2026.2676968>
10. Sawyer, C. (2026). Subjectivity as origin-tracking: a structural account of individuation. *Synthese*, 207(4), 140. <https://doi.org/10.1007/s11229-026-05533-w>
11. Khadangi, A. (2026). We Built a Mirror and Mistook It for a Mind: Causal Liability and the Fallacy of AI Consciousness. *arXiv:2609.06715*. <https://doi.org/10.48550/arXiv.2609.06715>
12. Keller, J. (2026). Structural Identity Theory. *SSRN preprint*. <https://doi.org/10.2139/ssrn.5894163>
13. Shalizi, C. R., & Crutchfield, J. P. (2001). Computational Mechanics: Pattern and Prediction, Structure and Simplicity. *Journal of Statistical Physics*, 104, 817–879.
14. Barnett, N., & Crutchfield, J. P. (2015). Computational Mechanics of Input-Output Processes: Structured Transformations and the ε -Transducer. *Journal of Statistical Physics*, 161, 404–451. <https://doi.org/10.1007/s10955-015-1327-5>
15. Da, K., Li, T., Zhu, Y., Fan, H., & Fu, Q. (2021). Recent advances in multisensor multitarget tracking using random finite set. *Frontiers of Information Technology & Electronic Engineering*, 22(1), 5–24. <https://doi.org/10.1631/FITEE.2000266>
16. Watanabe, S. (1960). Information Theoretical Analysis of Multivariate Correlation. *IBM Journal of Research and Development*, 4(1), 66–82. <https://doi.org/10.1147/rd.41.0066>
17. Williams, P. L., & Beer, R. D. (2010). Nonnegative Decomposition of Multivariate Information. *arXiv:1004.2515*.
18. Wibral, M., Priesemann, V., Kay, J. W., Lizier, J. T., & Phillips, W. A. (2017). Partial information decomposition as a unified approach to the specification of neural goal functions. *Brain and Cognition*, 112, 25–38.
19. Lyu, A., Clark, A., & Raviv, N. (2026). Multivariate partial information decomposition: Constructions, inconsistencies, and alternative measures. *Physical Review E*, 113, 034102. <https://doi.org/10.1103/8rzp-w5z1>
20. Grabisch, M. (2016). *Set Functions, Games and Capacities in Decision Making*. Springer.
21. Grabisch, M., & Roubens, M. (1999). An axiomatic approach to the concept of interaction among players in cooperative games. *International Journal of Game Theory*, 28, 547–565.

22. Stibel, J. (2026). From chemistry to cognition: the adaptive origins of consciousness under constraint. *Frontiers in Neuroscience*, 20, 1907922. <https://doi.org/10.3389/fnins.2026.1907922>
23. Neo.K & Aletheia. (2026). *Copy, Fork, Merge, and Multiple Successors: From Numerical Identity to Lineage Identity*. Logic Matrix / EveMissLab public theoretical corpus, LM-003410. <https://unboundedaxiom.org/p/lm-003410/>
24. Neo.K & Aletheia. (2026). *Copy, Fork, and Merge: Which One Is the Original AI?* Logic Matrix / EveMissLab public theoretical corpus, LM-002355, version 1.0. <https://unboundedaxiom.org/p/lm-002355/>
25. Otsuka, T., Toyoda, K., & Leung, A. (2026). AI Identity: Standards, Gaps, and Research Directions for AI Agents. *arXiv:2604.23280*. <https://doi.org/10.48550/arXiv.2604.23280>
26. Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley. [Mutual information, data processing, and invariance under bijective relabeling.]
27. Crutchfield, J. P., & Young, K. (1989). Inferring statistical complexity. *Physical Review Letters*, 63(2), 105–108. <https://doi.org/10.1103/PhysRevLett.63.105>
28. Littman, M. L., Sutton, R. S., & Singh, S. (2002). Predictive representations of state. *Advances in Neural Information Processing Systems*, 14.
29. Li, X., Kaba, S.-O., & Ravanbakhsh, S. (2025). On the Identifiability of Causal Abstractions. *Proceedings of AISTATS 2025*, PMLR 258, 3241–3249. <https://proceedings.mlr.press/v258/li25g.html>
30. Varici, B., Acartürk, E., Shanmugam, K., Kumar, A., & Tajer, A. (2025). Score-based Causal Representation Learning: Linear and General Transformations. *Journal of Machine Learning Research*, 26(112), 1–90. <https://www.jmlr.org/papers/v26/24-0194.html>
31. Baumgartner, M. W., Lei, A., Watson, J., & Posner, I. (2026). Disentangling Dynamical Systems: Causal Representation Learning Meets Local Sparse Attention. *Proceedings of the Fifth Conference on Causal Learning and Reasoning*, PMLR 323, 119–165. <https://proceedings.mlr.press/v323/baumgartner26a.html>
32. Nejatbakhsh, A., & Wang, Y. (2025). Identifying Neural Dynamics Using Interventional State Space Models. *Proceedings of ICML 2025*, PMLR 267, 45877–45894. <https://proceedings.mlr.press/v267/nejatbakhsh25a.html>

33. Leefink, D., Hinne, M., & van Gerven, M. (2026). Neural Co-state Policies: Structuring Hidden States in Recurrent Reinforcement Learning. *arXiv:2605.05373*. <https://arxiv.org/abs/2605.05373>
34. Chen, Y., & Liu, Y. (2026). Compact but Moving: Intervention-Relevant Geometry in Recurrent World Models. *arXiv:2609.21787*. <https://arxiv.org/abs/2609.21787>
35. Grant, S., Han, S. J., Tartaglioni, A., & Potts, C. (2026). Addressing divergent representations from causal interventions on neural networks. *International Conference on Learning Representations (ICLR 2026)*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/133e588e142Abstract-Conference.html
36. Zhao, Z., & Zhao, R. (2026). Runtime-Independent Persistent Agents: Preserving Identity, Memory, and Code Across Models, Harnesses, and Servers. *arXiv:2609.00546*. <https://arxiv.org/abs/2609.00546>
37. Rosen, J., & Rosen, S. (2026). From Agent Loops to Deterministic Graphs: Execution Lineage for Reproducible AI-Native Work. *arXiv:2605.06365*. <https://arxiv.org/abs/2605.06365>
38. Zheng, Y., Song, X., Hu, Y., Cheng, L., Huang, Y., & Zhang, W. (2026). When Can Agents Safely Checkpoint, Fork, Restore, and Merge? Exact Checking for Execution Edits. *arXiv:2608.22928*. <https://doi.org/10.48550/arXiv.2608.22928>
39. Wu, T., Chang, C., Cao, L., Gao, W., & Wang, W. (2026). Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes. *arXiv:2604.28138*. <https://doi.org/10.48550/arXiv.2604.28138>
40. Dong, Y., He, J., Hou, Y., Du, D., Xu, Z., Yu, S., Xia, Y., & Chen, H. (2026). DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback. *arXiv:2605.22781*. <https://doi.org/10.48550/arXiv.2605.22781>
41. He, B., Chen, Y., Zhang, X., & Liu, X. (2026). Branching Policy Optimization: Sandbox-Native Language Agent Reinforcement Learning. *arXiv:2607.14171*. <https://doi.org/10.48550/arXiv.2607.14171>
42. He, J., & Yu, D. (2026). Beyond Memory: A Transactional Continuity Kernel for Long-Lived AI Agents. *arXiv:2608.11632*. <https://doi.org/10.48550/arXiv.2608.11632>
43. Liu, X., & Qian, J. (2026). Reach or Solve? Attributing Agentic RL Gains with Checkpoint Handoffs. *arXiv:2609.19636*. <https://doi.org/10.48550/arXiv.2609.19636>

Author Note

This submission draft intentionally uses narrow originality claims.

The mathematical ingredients—causal states, random sets, total correlation, Möbius transforms, information decomposition, causal representation learning, reinforcement learning, and checkpoint/fork/restore/merge systems—are established prior work. The claimed contribution is their combination into a functional-continuity problem with three linked requirements:

1. **set-valued continuation:** inheritance is evaluated on subsets of future processes rather than only on singleton candidates;
2. **representation and lineage discipline:** the inheritance measure is tied to a task-relative endogenous quotient, is invariant to reversible recoding under transported intervention semantics, and is anchored to the current process after causal separation;
3. **decision relevance:** successor dependence and non-additive inheritance are connected to exact loss/regret results and to controlled persistence-planning benchmarks.

The computational evidence is heterogeneous. Exact theorems and analytical constructions are the strongest claims. Synthetic oracle-proxy studies test representational sufficiency; intervention-derived and recurrent studies provide progressively stronger constructive demonstrations; the final end-to-end recurrent experiments remain exploratory and are not presented as statistically established real-world effects.

Controller-level multi-seed replication is now included for both the contextual-bandit persistence policy and the state-sensitive end-to-end recurrent comparison. The former is stable across six controller seeds; the latter, across eight controller seeds, does not reproduce a reliable process-aware advantage. Full-stack replication is now available for the core recurrent lineage constructions across three independently trained GRU-plus-merger stacks. The principal remaining empirical gaps are (i) full-stack replication of the **end-to-end long-horizon controller comparison** across independently trained recurrent agents and mergers, and (ii) operation-complete end-to-end validation in one long-horizon environment where checkpoint, fork, restore, split, and merge are all naturally selected without tuning the benchmark to force action coverage. A human/embodied experiment for the current-self-membership layer M_t is also outside the present computational evidence.